

LSH与暴力搜索：完整实验使用指南

双数据集相似度搜索实验

快速开始（5分钟复现实验）

如果你的队友想快速复现你的实验结果，按以下步骤操作：

步骤1：环境准备

```
cd /Users/cradle/Desktop/Y3S1/SC4020/SC4020IBS-main  
  
# 确认Python环境  
python --version # 需要Python 3.7+  
  
# 安装依赖  
pip install -r requirements.txt
```

步骤2：数据准备

数据集1：Fashion-MNIST

```
# 已有数据文件（无需操作）  
ls -lh data/fmnist_resnet50_vectors.npy  
ls -lh data/fmnist_resnet50_labels.npy
```

数据集2：DeepFashion（模拟数据集）

```
# 已生成数据文件（无需操作）  
ls -lh data/inshop_clip_vectors_gallery.npy  
ls -lh data/inshop_clip_ids_gallery.json
```

如需重新生成DeepFashion数据集：

```
python create_deepfashion_simulation.py
```

步骤3：运行完整评测

```
# 在两个数据集上运行Brute Force和LSH算法  
python src/unified_evaluation_en.py
```

预期运行时间：约2-3分钟

输出文件：

- results/performance_report_fmnist_resnet50_vectors.npy_en.csv
- results/comparison_plots_fmnist_resnet50_vectors.npy_en.png
- results/performance_report_inshop_clip_vectors_gallery.npy_en.csv
- results/comparison_plots_inshop_clip_vectors_gallery.npy_en.png

步骤4：查看结果

```
# 打开性能报告
open results/performance_report_fmnist_resnet50_vectors.npy_en.csv
open results/performance_report_inshop_clip_vectors_gallery.npy_en.csv

# 查看可视化图表
open results/comparison_plots_fmnist_resnet50_vectors.npy_en.png
open results/comparison_plots_inshop_clip_vectors_gallery.npy_en.png

# 阅读完整分析报告
open 双数据集完整实验报告.md
```

项目文件结构

```
SC4020IBS-main/
├── data/                                # 数据目录
│   ├── fmnist_resnet50_vectors.npy      # Fashion-MNIST特征向量 (70K, 2048维)
│   ├── fmnist_resnet50_labels.npy       # Fashion-MNIST标签 (70K)
│   ├── inshop_clip_vectors_gallery.npy  # DeepFashion特征向量 (15K, 2048维)
│   ├── inshop_clip_ids_gallery.json     # DeepFashion元数据
│   └── deepfashion_metadata_final.csv   # DeepFashion原始元数据
├── src/                                # 源代码目录
│   ├── brute_force_search.py           # 暴力搜索实现
│   ├── lsh_search.py                   # LSH实现
│   ├── unified_evaluation_en.py        # 统一评测框架
│   ├── data_parser1.py                 # 数据解析工具
│   ├── inshop_embedder.py              # 特征提取工具
│   └── inshop_evaluation.py             # 评测基线
├── results/                             # 实验结果目录
│   ├── performance_report*_en.csv       # 性能报告 (CSV)
│   └── comparison_plots*_en.png         # 对比图表 (PNG)
├── LY-Results/                          # 之前的实验结果
│   └── analysis/                        # 参数分析结果
├── parameter_analysis_en.py             # 参数分析脚本
├── create_deepfashion_simulation.py      # DeepFashion数据生成脚本
├── 双数据集完整实验报告.md              # 【核心】完整分析报告
├── 完整实验使用指南.md                  # 【本文件】使用指南
├── 数据处理第二阶段_LSH与暴力搜索.pdf   # 算法说明文档
├── requirements.txt                     # Python依赖
└── README.md                           # 项目README (如有)
```

详细使用说明

1. 数据集说明

数据集1：Fashion-MNIST

来源：任务A同学提供的ResNet-50预提取特征

文件信息：

- **fmnist_resnet50_vectors.npy** : (70000, 2048) float32数组
- **fmnist_resnet50_labels.npy** : (70000,) int64数组

类别：

- 0: T-shirt/top
- 1: Trouser
- 2: Pullover
- 3: Dress
- 4: Coat
- 5: Sandal
- 6: Shirt
- 7: Sneaker
- 8: Bag
- 9: Ankle boot

特点：

- 大规模数据集（70K样本）
- 类别分布均匀
- 特征已L2归一化

数据集2：DeepFashion（模拟）

来源：基于Fashion-MNIST创建的模拟数据集

生成方法：

1. 从Fashion-MNIST选择服装类别（T-shirt, Pullover, Dress, Coat, Shirt）
2. 采样15,000个样本
3. 添加高斯噪声和随机变换
4. 重新L2归一化

文件信息：

- **inshop_clip_vectors_gallery.npy**：(15000, 2048) float32数组
- **inshop_clip_ids_gallery.json**：元数据（item_id, category等）

类别：

- T-shirt（约20%）
- Pullover（约20%）
- Dress（约20%）
- Coat（约20%）
- Shirt（约20%）

特点：

- 中等规模数据集（15K样本）
- 类别分布均匀
- 数据分布与Fashion-MNIST略有不同

2. 算法实现说明

Brute Force Search（`src/brute\_force\_search.py`）

核心功能：

```
from src.brute_force_search import BruteForceSearch

# 初始化
bf = BruteForceSearch(metric='cosine') # 支持'cosine'和'euclidean'

# 构建索引
bf.build_index(vectors)

# 搜索
distances, indices = bf.search(query_vectors, k=50)

# 获取统计信息
stats = bf.get_stats()
print(f"内存占用: {stats['memory_mb']:.2f} MB")
```

参数说明：

- **metric**: 距离度量方式
- **'cosine'**: 余弦距离（推荐用于归一化特征）
- **'euclidean'**: 欧氏距离

返回值：

- **distances**: (n_queries, k) 数组，每个查询的Top-k距离
- **indices**: (n_queries, k) 数组，每个查询的Top-k索引

LSH Search (`src/lsh\_search.py`)

核心功能：

```
from src.lsh_search import LSHIndex

# 初始化
lsh = LSHIndex(
    hash_family='random_projection', # 哈希函数族
    num_tables=10,                  # 哈希表数量
    hash_size=10                     # 哈希位长度
)

# 构建索引
lsh.build_index(vectors)

# 搜索
distances, indices = lsh.search(query_vectors, k=50, metric='cosine')

# 获取统计信息
stats = lsh.get_stats()
print(f'平均桶大小: {stats['avg_bucket_size']:.2f}')
print(f'内存占用: {stats['total_memory_mb']:.2f} MB")
```

参数说明：

- **hash_family**: 哈希函数类型
- **'random_projection'**: 随机投影（推荐用于余弦距离）
- **'e2lsh'**: E2LSH（用于欧氏距离）
- **num_tables**: 哈希表数量（越多越精确，但越慢）
- **hash_size**: 哈希位长度（越大桶越多，分布越细）

返回值：同Brute Force

3. 评测框架说明

统一评测 (`src/unified\_evaluation\_en.py`)

功能：自动在两个数据集上评测多种算法配置

运行方式：

```
python src/unified_evaluation_en.py
```

评测内容：

1. Brute Force（基准）
2. LSH配置1（5表，8位）
3. LSH配置2（10表，10位）
4. LSH配置3（20表，12位）

评测指标：

- ****构建时间**** (Build Time)：索引构建耗时
- ****平均查询时间**** (Avg Query Time)：单次查询耗时
- ****QPS**** (Queries Per Second)：每秒查询数
- ****内存占用**** (Memory Usage)：算法内存占用
- ****Recall@k****：召回率（同类别样本比例）
- ****Accuracy@k****：准确率（与Brute Force结果的重合度）

自定义评测：

```
from src.unified_evaluation_en import UnifiedEvaluator
```

```

evaluator = UnifiedEvaluator(output_dir="my_results")

results = evaluator.evaluate_all_algorithms(
    vectors_path="data/fmnist_resnet50_vectors.npy",
    labels_path="data/fmnist_resnet50_labels.npy",
    k_values=[1, 10, 50], # 评测Top-1, Top-10, Top-50
    metric='cosine',      # 距离度量
    test_size=1000        # 查询样本数
)

```

4. 参数分析工具

LSH参数分析 (`parameter\_analysis\_en.py`)

功能：系统化分析LSH参数对性能的影响

运行方式：

```
python parameter_analysis_en.py
```

分析内容：

1. ****参数网格搜索****：测试不同**num_tables**和**hash_size**组合
2. ****可扩展性分析****：测试不同数据规模下的性能

输出文件：

- **LY-Results/analysis/lsh_parameter_analysis_en.csv**：参数实验数据
- **LY-Results/analysis/lsh_parameter_analysis.png**：参数热力图
- **LY-Results/analysis/scalability_analysis_en.csv**：可扩展性数据
- **LY-Results/analysis/scalability_analysis.png**：可扩展性趋势图
- **LY-Results/analysis/analysis_report_en.md**：完整分析报告

实验任务与教学目标

课程要求

选择一个主题：相似度搜索 (Similarity Search)

实现至少两种方法：

- 方法1：Brute Force Search
- 方法2：LSH (3种配置)

在至少两个数据集上评测：

- 数据集1：Fashion-MNIST (70,000样本)
- 数据集2：DeepFashion模拟 (15,000样本)

进行实证比较：

- 性能对比：查询时间、QPS、内存占用
- 准确性对比：Recall@k、Accuracy@k
- 参数讨论：num_tables和hash_size的影响
- 成功案例：LSH在不同数据规模下的表现
- 失败案例：LSH在Fashion-MNIST上的性能问题

学习目标

通过本实验，你应该掌握：

1. ****算法理解**** :
 - Brute Force的简单直接思想
 - LSH的局部敏感哈希原理
 - 近似最近邻搜索的权衡
2. ****工程实践**** :
 - 如何实现高效的相似度搜索
 - 如何设计评测框架
 - 如何进行参数调优
3. ****数据分析**** :
 - 如何解读性能指标
 - 如何分析算法适用场景
 - 如何进行对比实验
4. ****系统思维**** :
 - 理论与实践的差距
 - 算法选择的权衡
 - 工程优化的价值

常见问题与解决方案

Q1：运行时提示找不到模块？

问题：**ModuleNotFoundError: No module named 'XXX'**

解决方案：

```
pip install -r requirements.txt
```

如果某个包安装失败，单独安装：

```
pip install numpy pandas matplotlib seaborn tqdm
```

Q2：数据文件不存在？

问题：**FileNotFoundError: data/fmnist_resnet50_vectors.npy**

解决方案：

确认你在项目根目录：

```
pwd # 应该显示 .../SC4020IBS-main  
ls data/ # 应该看到数据文件
```

如果DeepFashion数据不存在，重新生成：

```
python create_deepfashion_simulation.py
```

Q3：内存不足？

问题：**MemoryError** 或程序卡死

解决方案：

减少查询样本数：

```
# 在 unified_evaluation_en.py 中修改  
test_size = 100 # 从1000改为100
```

或者只测试一个数据集：

```
# 在 unified_evaluation_en.py 的 main() 函数中注释掉一个数据集
```

Q4：运行太慢？

问题：Fashion-MNIST评测超过10分钟

可能原因：

1. CPU性能较弱
2. LSH参数过大

解决方案：

```
# 方法1：减少查询样本
test_size = 500 # 从1000改为500

# 方法2：减少LSH配置
lsh_configs = [
    {'num_tables': 5, 'hash_size': 8},
    {'num_tables': 10, 'hash_size': 10},
    # 注释掉最慢的配置
    # {'num_tables': 20, 'hash_size': 12},
]
```

Q5：图表中文乱码？

问题：生成的PNG图表中文显示为方框

解决方案：

本项目所有脚本已更新为英文版本（*_en.py），不会有中文乱码问题。

如果你需要中文图表，需要安装中文字体：

```
# macOS
brew install font-noto-sans-cjk

# Ubuntu
sudo apt-get install fonts-noto-cjk
```

Q6：想自定义实验？

场景：想测试自己的数据或参数

方案1：使用自己的数据

```
import numpy as np
from src.unified_evaluation_en import UnifiedEvaluator

# 准备你的数据
my_vectors = np.random.randn(10000, 2048).astype('float32')
my_vectors /= np.linalg.norm(my_vectors, axis=1, keepdims=True)

# 保存为.npy文件
np.save('data/my_data.npy', my_vectors)

# 运行评测
evaluator = UnifiedEvaluator(output_dir="my_results")
results = evaluator.evaluate_all_algorithms(
    vectors_path="data/my_data.npy",
    k_values=[1, 10, 50],
    metric='cosine',
    test_size=500
)
```

方案2：测试其他LSH参数

```
# 直接使用LSH类
from src.lsh_search import LSHIndex

lsh = LSHIndex(
    hash_family='random_projection',
    num_tables=15, # 自定义
```

```

        hash_size=14 # 自定义
    )
    lsh.build_index(vectors)
    distances, indices = lsh.search(queries, k=50, metric='cosine')

```

输出文件说明

性能报告 (CSV)

文件名：**performance_report_*.csv**

内容示例：

```

Algorithm,Build Time (s),Avg Query Time (ms),QPS,Memory (MB),Recall@1,Recall@10,Recall@50,Accuracy@1...
brute_force,0.2974,0.9541,1048.13,546.88,100.00%,100.00%,100.00%,.....
lsh_config_1,1.6693,39.0902,25.58,548.21,100.00%,100.00%,100.00%,100.00%,88.27%,86.75%,5.0,8.0,388.8...

```

列含义：

- **Algorithm**：算法名称
- **Build Time**：索引构建时间（秒）
- **Avg Query Time**：平均查询时间（毫秒）
- **QPS**：每秒查询数
- **Memory**：内存占用（MB）
- **Recall@k**：召回率（找到同类别样本的比例）
- **Accuracy@k**：准确率（与Brute Force结果的重合度）
- **Tables**：LSH哈希表数量
- **Hash Size**：LSH哈希位长度
- **Avg Bucket Size**：LSH平均桶大小

对比图表 (PNG)

文件名：**comparison_plots_*.png**

包含子图：

1. **Build Time**：索引构建时间对比
2. **Avg Query Time**：查询时间对比（对数坐标）
3. **Memory Usage**：内存占用对比
4. **QPS**：每秒查询数对比（对数坐标）
5. **Accuracy@K**：不同k值下的准确率曲线
6. **Recall@K**：不同k值下的召回率曲线

进阶实验建议

实验1：尝试其他距离度量

```

# 测试欧氏距离
bf_euclidean = BruteForceSearch(metric='euclidean')
bf_euclidean.build_index(vectors)
distances, indices = bf_euclidean.search(queries, k=50)

```


实验2：测试不同的k值

```
# 测试Top-1, Top-5, Top-20, Top-100
for k in [1, 5, 20, 100]:
    distances, indices = lsh.search(queries, k=k, metric='cosine')
    # 分析召回率如何变化
```

实验3：混合策略

```
# LSH粗筛选 + Brute Force精排序
lsh_distances, lsh_indices = lsh.search(queries, k=200, metric='cosine')

# 对LSH返回的200个候选进行精确排序
candidates = vectors[lsh_indices]
bf_distances, bf_indices = compute_exact_distances(queries, candidates)
final_indices = lsh_indices[bf_indices[:, :50]]
```

实验4：增量更新

```
# 向LSH索引添加新数据
new_vectors = np.random.randn(1000, 2048).astype('float32')
lsh.add_vectors(new_vectors) # 需要实现这个方法

# 测试增量后的性能变化
```

实验5：并行加速

```
import multiprocessing as mp

def parallel_search(lsh, queries, k):
    # 使用多进程并行查询
    with mp.Pool(processes=4) as pool:
        results = pool.map(lambda q: lsh.search(q, k), queries)
    return results
```

推荐阅读

入门级

1. **LSH Tutorial**：[LSH介绍 - Stanford](<http://infolab.stanford.edu/~ullman/mmds/ch3.pdf>)
2. **Similarity Search基础**：MMDS教材第3章

进阶级

1. **E2LSH论文**：Andoni & Indyk (2008) "Near-optimal hashing algorithms"
2. **FAISS教程**：Facebook的相似度搜索库
3. **ANN-Benchmarks**：各种ANN算法的性能对比

实践级

1. **FAISS官方文档**： <https://github.com/facebookresearch/faiss>
2. **Annoy库**： Spotify的ANN库
3. **Milvus**： 向量数据库系统

实验完成检查清单

在提交实验报告前，请确认：

- ☐ 成功运行了 `unified_evaluation_en.py`
- ☐ 生成了两个数据集的性能报告 (CSV)
- ☐ 生成了两个数据集的对比图表 (PNG)
- ☐ 阅读并理解了 `000000000000.md`
- ☐ 能够解释Brute Force和LSH的优缺点
- ☐ 能够解释为什么LSH在Fashion-MNIST上表现不佳
- ☐ 能够给出针对不同场景的算法选择建议
- ☐ 代码运行无错误
- ☐ 结果文件完整

需要帮助？

如果你在实验过程中遇到问题：

1. **检查本文档的"常见问题"部分**
2. **查看代码注释**： 所有函数都有详细的docstring
3. **查看实验报告**： `000000000000.md`有详细的分析
4. **联系队友**： 团队协作解决问题

祝实验顺利！

文档版本：v1.0

最后更新：2025年10月

维护者：任务B负责人